

T-Control: An Efficient Dynamic Tensor Rematerialization System for DNN Training

Zehua Wang

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
wangzehua22s@ict.ac.cn

Junmin Xiao*

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
xiaojunmin@ict.ac.cn

Xiaochuan Deng

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
dengxiaochuan24@mails.ucas.ac.cn

Huiping Wang

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
wanghuiping23s@ict.ac.cn

Hui Ma

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
mahui@ncic.ac.cn

Mingyi Li

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
limingyi@ncic.ac.cn

Yunfei Pang

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
pangyunfei@ict.ac.cn

Guangming Tan*

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China
tgm@ict.ac.cn

Abstract

With the continuous growth of model and batch sizes, DNN model training increasingly suffers from excessive memory consumption. Tensor rematerialization has emerged as an effective technique to enable training under limited memory constraints. However, dynamic rematerialization methods often underperform static approaches, primarily due to their greedy tensor eviction schedules and runtime overhead. In this work, we develop T-Control, a dynamic tensor rematerialization system for DNN model training. T-Control integrates the topology of the traced tensor dependency graph with real-time memory usage to make informed, adaptive tensor retention decisions, preserving critical tensors and reducing eviction-induced recomputation. Furthermore, by extending PyTorch's native memory manager with our fine-grained memory management strategy, T-control effectively improves memory utilization, thereby reducing unnecessary tensor eviction. Experimental results demonstrate that T-Control boosts throughput by up to 1.58 $\times$ and 1.91 $\times$ over state-of-the-art static and dynamic tensor rematerialization systems.

*Junmin Xiao and Guangming Tan are corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2359-9/2026/03

<https://doi.org/10.1145/3779212.3790230>

CCS Concepts: • Computer systems organization → Real-time operating systems; • Computing methodologies → Machine learning.

Keywords: Tensor Rematerialization; Memory Management; GPU; Deep Learning

ACM Reference Format:

Zehua Wang, Junmin Xiao, Xiaochuan Deng, Huiping Wang, Hui Ma, Mingyi Li, Yunfei Pang, and Guangming Tan. 2026. T-Control: An Efficient Dynamic Tensor Rematerialization System for DNN Training. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '26)*, March 22–26, 2026, Pittsburgh, PA, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3779212.3790230>

1 Introduction

In recent years, deep learning has gained unprecedented success in various domains [7, 28, 52]. With the increasing scale of deep neural network (DNN) models [1, 40], the memory demand for model training has been growing dramatically.

To enable training under restricted memory resources, the *Tensor Rematerialization* (TR) technique has been developed, which follows the principle of releasing the memory of certain activations during forward propagation and re-computing them during backward propagation when needed [52]. This strategy allows training with models and batches that exceed the capacity of on-device memory without modifying the model architecture, and has therefore attracted considerable attention [4, 9, 17, 20, 24, 25, 38, 45, 50].

Existing TR approaches can generally be categorized into static methods and dynamic methods: static methods determine the recomputation schedule in advance based on the computational graph [4, 9, 20, 25, 38, 45, 51], providing predictable performance but limited adaptability, whereas dynamic methods adjust rematerialization decisions at runtime [17, 24, 45, 50], offering greater flexibility at the cost of additional runtime overhead. Particularly, dynamic methods are naturally well-suited for training dynamic models, as they adapt decisions at runtime without requiring a fixed computation graph. By contrast, static methods often necessitate unrolling the dynamic structure and replanning for each input, which is prohibitively expensive. Thus, existing static TR methods/systems do not support dynamic models.

While dynamic methods offer clear advantages for dynamic models, static methods generally achieve higher performance in practice for two reasons. First, static methods construct the entire Tensor Dependency Graph (TDG) in advance and solve a global optimization problem to generate well-informed tensor-eviction plans [4, 20], whereas dynamic methods rely on runtime-cost models and may greedily evict hub tensors, incurring excessive rematerialization overhead [24, 45]. Second, static methods follow fixed memory access patterns that simplify fragmentation management [14], while the highly variable access patterns of dynamic methods exacerbate fragmentation [23], triggering further tensor eviction and recomputation [50].

To overcome the bottlenecks of existing dynamic methods, we review the implementations of dynamic TR for the training of different DNN models, and find two key opportunities for performance improvement. First, by integrating the topology of a traced partial TDG with real-time memory usage, dynamic methods can make informed, memory-adaptive tensor retention decisions, thereby preserving critical tensors and reducing unnecessary evictions. Second, adaptively adjusting tensor placement in memory can effectively reuse memory fragments and minimize tensor eviction, fully exploiting memory. Motivated by these observations, we develop T-Control, a novel dynamic tensor rematerialization system that enables DNN training with limited memory budgets. T-Control integrates a dynamic tensor retention algorithm to preserve critical tensors and a fine-grained memory management strategy to guide tensor allocation, migration, and eviction, enhancing performance and memory efficiency.

T-Control incorporates two key features that distinguish it from existing works. (1) It enables informed tensor eviction decisions by leveraging the topological structure of the traced TDG at runtime, rather than relying solely on tensor-specific attributes such as tensor size and computation cost. This ensures that critical hub tensors are preserved in memory, thereby avoiding deep recursive recomputation. (2) It augments PyTorch’s native memory manager with our developed tensor management strategy that adaptively adjusts tensor placement in memory to improve the reuse

of fragmented memory regions, thereby reducing unnecessary tensor eviction. Consequently, by tightly integrating the dynamic tensor retention/eviction with our fine-grained memory management functionality, T-Control achieves high performance comparable to state-of-the-art static methods. Specifically, our contributions can be summarized as follows:

- Propose T-Control, an efficient dynamic tensor rematerialization system for DNN training (Section 3), based on an in-depth analysis of existing dynamic methods.
- Design a dynamic tensor retention algorithm to maximize memory utilization and reduce rematerialization overhead (Section 4), and develop a fine-grained memory management strategy to reduce memory fragmentation (Section 5).
- Evaluate and analyze the performance of T-Control, and prove that the proposed system T-Control boosts throughput by up to 1.58× and 1.91× over the state-of-the-art static and dynamic TR systems (Section 6).

2 Motivation

2.1 Tensor Rematerialization Technique

Tensor dependency graph. As shown in Figure 1, the *tensor dependency graph* (TDG) is a directed graph $G(V, E)$, where V is the set of vertices representing tensors, and $E \subseteq V \times V$ is the set of directed edges encoding dependencies between tensors. Each vertex v is annotated with a four-tuple attribute: (1) **Staleness** $st(v)$, indicating the time elapsed since the tensor was last accessed, (2) **Memory** $m(v)$, representing the tensor size, (3) **Computation cost** $c(v)$, the time required to calculate v from its parent tensors based on its computational path, and (4) **Eviction status** $e(v)$, a flag indicating whether v has been evicted. For each directed edge $(u, v) \in E$, v and u are called its *head* and *tail* respectively, and v depends on u , meaning that v cannot be computed without u [22].

Tensor rematerialization. To train large models or batches beyond limited memory without altering models, tensor rematerialization (TR) is widely adopted [48], which is classified into *static methods* [4, 9, 10, 20] and *dynamic methods* [17, 24, 50]. Both methods release selected intermediate tensors during forward propagation and recompute them in backward propagation when needed (Figure 1). They differ in the timing of tensor eviction decisions. Static methods establish the eviction plan before training based on TDG, while dynamic methods decide at runtime using cost models.

2.2 Analysis of Dynamic Methods

For DNN training, dynamic methods are usually hard to achieve high performance comparable to static methods (Figure 2(a)), primarily due to two inherent limitations.

Limitation 1: Higher costs for dynamic TR. The existing dynamic methods typically involve more frequent tensor rematerializations and deeper recursion in dynamic rematerialization (Figure 2(b) and (c)). Specifically, dynamic methods

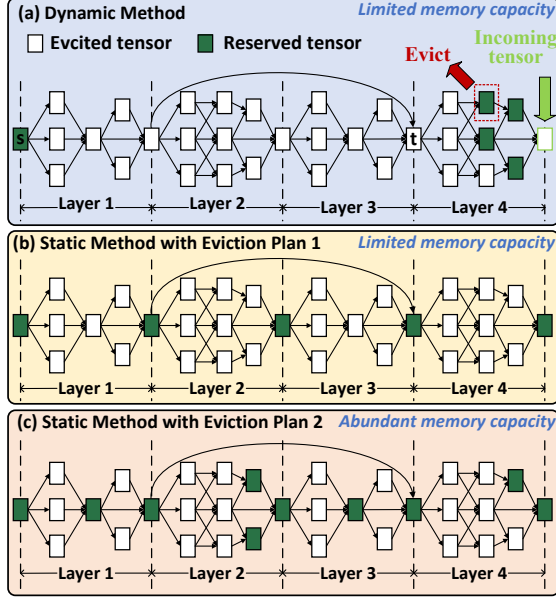


Figure 1. An example of tensor rematerialization methods. In Panel (a), rematerializing the evicted tensor t requires up to 9 computation recursions from the reserved tensor s .

perform greedy decision making at runtime [24], which not only incurs additional cost, but also tends to retain newly generated tensors while evicting tensors unused in memory for a prolonged period (Figure 1(a)). This eviction focuses primarily on tensor-specific properties but neglects the dependency relationships among tensors, which may release critical tensors serving as hubs in TDGs, typically resulting in massive rematerialization events and deep computation recursions [45]. By contrast, static methods leverage the entire TDG to predetermine tensor eviction plans before runtime, minimizing TR overhead. As shown in Figure 1(b), a static method retains tensors at regular intervals, effectively reducing rematerialization events and deep recursion.

Limitation 2: More severe memory fragmentation under dynamic TR. As illustrated in Figure 3(a), both static and dynamic rematerialization methods suffer from increased memory fragmentation as the memory budget decreases. However, due to more rematerialization events (Figure 2(b)), dynamic methods (e.g., DTR) exhibit significantly higher fragment ratios compared to static methods (e.g., Megatron-LM), sometimes exceeding 25% of the total memory footprint.

Moreover, recent systems like GMLake [14] introduce virtual memory stitching (VMS) to better utilize idle memory fragments. While VMS effectively reduces fragmentation, the construction of virtual memory mapping incurs high overhead. This makes it suitable primarily for static methods, where memory usage patterns remain stable across iterations, allowing the virtual memory mapping to be built once

and reused. In contrast, dynamic methods such as DTR [24] frequently alter memory usage, requiring the mapping to be reconstructed at every iteration, which severely impacts training efficiency. As shown in Figure 3(b), applying GMLake to DTR successfully reduces the gap between the peak allocated memory and reserved memory, but it significantly degrades performance.

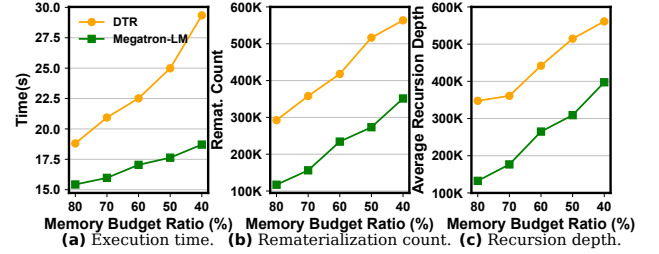


Figure 2. Execution time, rematerialization count, and average recursion depth of each TR per iteration in the training of GPT 3-7.5B. Here, DTR [24] and Megatron-LM [31] represent the state-of-the-art dynamic and static methods respectively.

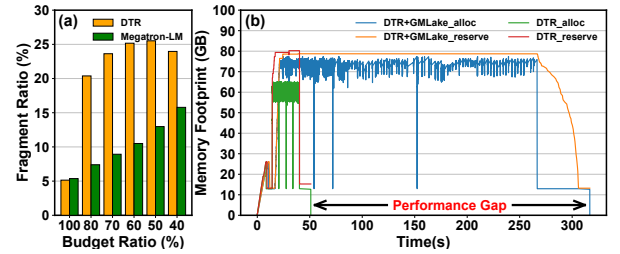


Figure 3. Memory usage. Panel (a) presents the fragment ratio of DTR and Megatron-LM in the training of GPT 3-7.5B. Panel (b) shows the memory footprint per iteration of Llama 2-7B Lora fine-tuning. The *alloc* and *reserve* mean the peak allocated memory and peak reserved memory respectively.

2.3 Key Ideas

While existing dynamic methods exhibit certain limitations, they have the potential to achieve performance comparable to that of static methods. Motivated by this prospect, our work is driven by two key ideas.

Key Idea 1. Integrating TDG topology with real-time memory usage enables topology-aware tensor retention, avoiding deep recursive recomputation and unnecessary tensor evictions.

This idea builds on two observations. First, the topology of the traced TDG provides a natural basis for assessing tensor importance. As shown in Figure 1, tensors with high dependency centrality, such as layer inputs and outputs, serve as key dataflow hubs that are depended upon by many downstream operations. Evicting these tensors triggers excessive recomputation and deep recursive rematerialization. Therefore, exploiting the TDG topology [16] allows dynamic methods to

identify and prioritize the retention of structurally important tensors, forming a more informed and dependency-aware TR schedule. Second, *the dynamic coordination of tensor retention and eviction enables efficient exploitation of runtime memory utilization and avoids unnecessary evictions*. By monitoring memory footprint at runtime, dynamic methods can jointly adjust tensor retention and eviction decisions to exploit transient memory availability and fragmentation across batches. As a result, different batches may adopt distinct TR plans (e.g., Figure 1(b) and (c)), allowing more tensors to be retained and reducing eviction frequency. In contrast, predetermined and batch-invariant static schedules conservatively assume worst-case memory usage and are often constrained to adopt restrictive plans such as Figure 1(b), missing opportunities for more efficient alternatives even when sufficient memory is temporarily available.

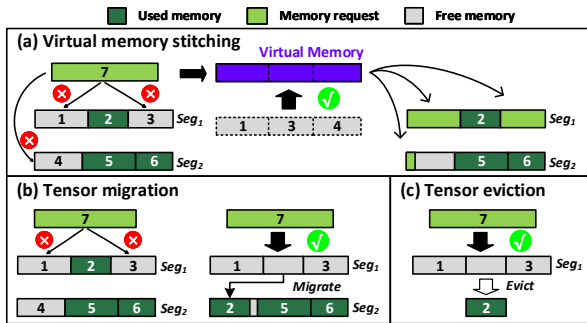


Figure 4. Different approaches to reuse memory fragments.

Key Idea 2. *Adaptively adjusting tensor placement in memory enables more efficient reuse of available memory and reduces unnecessary tensor evictions.*

This idea is also based on two observations. First, *relocating tensors within memory could achieve a similar effect as VMS in reusing memory fragments, but with significantly lower overhead, making it practical for dynamic TR*. As illustrated in Figure 4(a), VMS maps Block 7 onto a stitched region formed by Blocks 1, 3, and 4 via a virtual address, allowing Block 7 to occupy physically non-contiguous chunks and thereby reducing fragmentation. However, constructing such mappings repeatedly incurs high runtime overhead, which is impractical for dynamic TR. In contrast, as shown in Figure 4(b), tensor relocation moves Block 2 into the space of Block 4, creating a contiguous region sufficient to accommodate Block 7, while simultaneously reducing fragmentation. Furthermore, tensor relocation reuses memory fragments at negligible cost, on the order of microseconds. Second, *runtime tensor migration serves as a proactive mechanism to avoid tensor eviction*. Based on the runtime memory footprint, dynamic TR methods can proactively migrate tensors to merge fragmented regions into larger contiguous segments, enabling successful allocations without eviction or recomputation. However, existing static systems without

VMS, such as Megatron-LM [31] and AdaPipe [38], must evict tensors once contiguous space becomes insufficient, as shown in Figure 4(c).

3 System Overview

Based on the two key ideas above, we propose T-Control, a dynamic tensor rematerialization system. We implement a prototype with lightweight modifications to the PyTorch framework [27], incorporating tensor retention capabilities and fine-grained memory management. The system architecture is illustrated in Figure 5, where T-Control consists of four components:

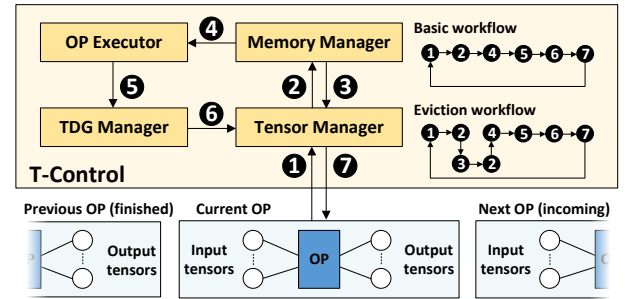


Figure 5. The system architecture of T-Control.

OP Executor executes individual operators with the provided tensors, which delegates both computation and data acquisition to the underlying PyTorch engine.

Memory Manager handles memory requests based on a fine-grained memory management strategy that promotes contiguous memory usage (Section 5).

Tensor Manager maintains metadata for all the tensors involved in training and encapsulates them into system-managed objects, supporting allocation, migration, eviction.

TDG Manager dynamically constructs and maintains TDG, implementing the tensor retention algorithm (Section 4).

In T-Control, these four components operate cooperatively at the granularity of individual operators and follow two workflows, as illustrated in Figure 5.

Basic workflow: The basic workflow represents T-Control’s standard runtime path, integrating both retention and memory optimizations. In this workflow, T-Control executes the following steps: (1) intercepting each operator together with its input tensors and passing them to the **Tensor Manager** for preprocessing ①; (2) processing allocation requests for the required output tensors ②; (3) delivering the output tensor handles, input tensors, and operators to the **OP Executor** for actual computation ④. After execution, (4) the **TDG Manager** updates the dependency relationships between input and output tensors in the TDG ⑤; and (5) the **Tensor Manager** performs postprocessing on the output tensors ⑥ (e.g., marking for future eviction or locking), before returning them to the original training process ⑦. Additionally,

after a new layer is completed, the **TDG Manager** may issue lock or unlock instructions for tensors selected to be retained during the postprocessing stage ⑥.

Eviction workflow: The eviction workflow is triggered when memory usage approaches the predefined budget. In this case, eviction instructions are sent to the **Tensor Manager** for execution ③, after which the workflow proceeds identically to the basic workflow.

4 Dynamic Tensor Retention Algorithm

To avoid frequent tensor rematerializations and deep recursion [8] in dynamic TR, we quantify the tensor importance based on the topology of TDG and selectively retain relatively important tensors.

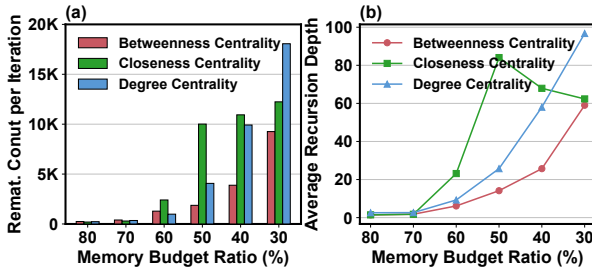


Figure 6. The rematerialization count and average recursion depth of each TR in the Llama 2-7B training, by retaining tensors based on the top 5% of their centrality scores.

4.1 Quantitation of Vertex Importance

Selection of metrics for quantifying vertex importance.

In complex network theory, vertex centrality metrics, such as betweenness centrality (BC) [5, 12], closeness centrality, and degree centrality [3, 36], are commonly used to quantify the extent to which a vertex is relied upon by others. Figure 6 compares the rematerialization counts and average recursion depth of each TR during Llama 2-7B training when we preserve tensors corresponding to the top 5% of vertices ranked by each centrality metric. Across different memory budget ratios, BC consistently results in the lowest rematerialization frequency and recursion depth among the evaluated metrics. Based on these observations, T-Control adopts BC as the metric for quantifying vertex importance in the TDG.

Betweenness centrality based on shortest computation paths. Given a TDG $G(V, E)$, a directed path $p_{(s,t)}$ from s to t is an alternating sequence of vertices and edges $\{s, (s, v_1), v_1, (v_1, v_2), v_2, \dots, (v_k, t), t\}$ starting with $s \in V$ and ending with $t \in V$, such that the vertices before and after an edge are its tail and head. The length of $p_{(s,t)}$ is defined as the total computation cost of calculating t from s along the path: $\text{length}(p_{(s,t)}) = \sum_{1 \leq i \leq k} c(v_i) + c(t)$, where $c(v_i)$ and $c(t)$ represent the computation costs of v_i and t , respectively.

The distance $\text{dist}(s, t)$ from s to t is the minimum length of any directed path if one exists, and undefined otherwise. A (s, t) -shortest computation path (SCP) is any directed path from s to t whose length equals $\text{dist}(s, t)$.

Let $\sigma(s, t)$ be the number of (s, t) -SCPs, and denote $\sigma(s, t; v)$ as the number of (s, t) -SCPs that pass through a vertex v , other than s or t . Similar to the standard definition of BC [6, 12], the SCP-based BC of a vertex $v \in V$ is defined as:

$$BC_{SCP}(v) = \sum_{s,t \in V} b(s, t; v), \text{ where } b(s, t; v) = \frac{\sigma(s, t; v)}{\sigma(s, t)}. \quad (1)$$

Here, $BC_{SCP}(v)$ quantifies the importance of v as an intermediary in computational dependencies, which can be interpreted as the degree to which a vertex v has control over pair-wise connections between other vertices s and t . From Figure 7, it is clear that the hub vertices in TDG have higher $BC_{SCP}(v)$ compared with the others.

4.2 Incremental SCP-based BC Update

While SCP-based BC effectively captures the structural importance of vertices in the TDG, its practical utility depends on efficiently computing SCP-based BC scores for all vertices at runtime as the TDG expands. The key challenge lies in updating SCP-based BC values on a growing TDG without recomputing them from scratch. To address this, we develop an incremental SCP-based BC update approach that enables updating vertex BC values during TDG construction.

Our design is informed by prior advances in *incremental processing* for dynamic and streaming graphs [29, 30, 42, 43]. First, an inter-snapshot incremental processing is proposed to amortize data fetching and computation costs across evolving graph snapshots [42]. This motivates us to reuse SCP-based BC values computed on earlier traced TDG when processing newly expanded TDG. Second, a series of streaming graph optimization frameworks has emerged over the past decade. KickStarter [43] enables runtime incremental updates through trimmed approximations; GraphBolt [30] extends this approach with a dependency-driven incremental model that preserves BSP semantics; and DZiG [29] further incorporates sparsity-aware update pruning. Inspired by these systems, we adopt the incremental processing paradigm and tailor it to runtime SCP-based BC computation on the traced TDG.

Unlike existing solutions designed for large-scale, arbitrarily evolving graphs [29, 30], the TDG in our setting is of medium scale (typically comprising thousands of vertices and edges) and evolves with a highly regular pattern determined by the layered structure of DNNs. By leveraging this structural regularity, we design a customized and direct algorithm that efficiently and accurately computes BC scores during DNN training, eliminating the need for iterative corrections that are commonly required by existing streaming graph systems [43].

Hierarchical perspective on TDG. By analyzing the topologies of TDGs traced in DNN training, we identify an inherent two-level hierarchical structure, as shown in Figure 7.

At the lower level, TDG exhibits a subgraph sequence structure, where the i -th subgraph $G_i(V_i, E_i)$ consists of all tensors involved in the i -th neural network layer ($i \geq 1$). Each subgraph G_i has a single input vertex t_{in}^i and a single output vertex t_{out}^i , corresponding to the input and output tensors of the i -th layer, respectively. The output vertex of G_i serves as the input vertex for G_{i+1} , forming a chain of layer-wise dependencies. For layers with multiple inputs/outputs, we concatenate them into a single input/output representation.

At the upper level, the TDG captures a skip-connection structure $G_0(V_0, E_0)$, where V_0 includes inputs t_{in}^i and outputs t_{out}^i of all the subgraphs, and each shortcut edge in E_0 bypasses multiple subgraphs, representing long-range inter-layer dependencies introduced by architectural constructs such as residual and non-local connections [15, 44].

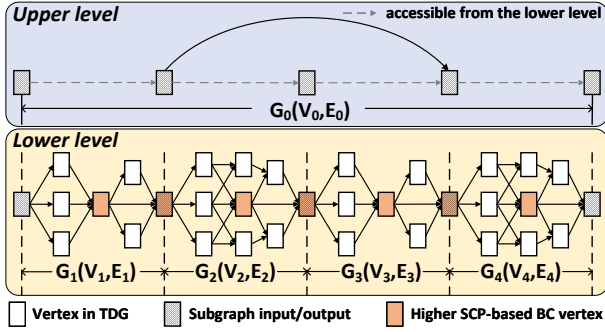


Figure 7. Two-level hierarchy of the TDG in Figure 1.

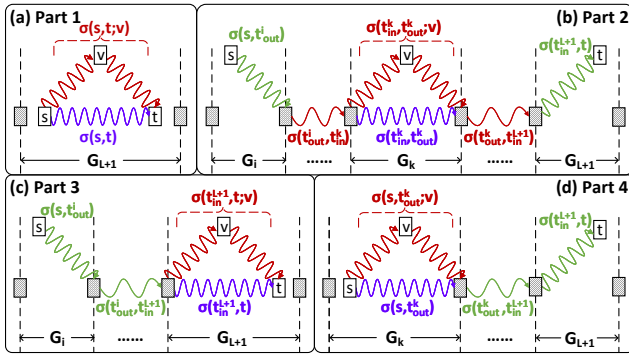


Figure 8. Decomposition of SCP-based BC into four parts. $\lambda(v)$, $\alpha(v)$, $\beta(v)$, and $\gamma(v)$ are incremental terms for them.

Decomposition of SCP-based BC. During DNN training, T-Control traces the TDG incrementally, extending the graph each time a new layer is executed. At any point, the currently traced TDG can be expressed as $G(V, E) = \bigcup_{k=0}^{L+1} G_k(V_k, E_k)$, where $G_{L+1}(V_{L+1}, E_{L+1})$ is the newly generated subgraph.

Based on the subgraph sequence structure at the lower level of TDG, we decompose $BC_{SCP}(v)$ for a vertex $v \in V_k$ into the following four disjoint parts, as illustrated in Figure 8.

Part 1 is $\sum_{s,t \in V_k} b(s, t; v)$, which accounts for SCPs in G_k .

Part 2 is $\sum_{i < k < j} \sum_{s \in V_i, t \in V_j} b(s, t; v)$, which focuses on SCPs that connect two different G_i and G_j , passing through G_k .

Part 3 is $\sum_{i < k} \sum_{s \in V_i, t \in V_k} b(s, t; v)$, which captures incoming SCPs where s originates from G_i , while t terminates in G_k .

Part 4 is $\sum_{j > k} \sum_{s \in V_k, t \in V_j} b(s, t; v)$, which accounts for the outgoing paths where s belongs to G_k and t resides in G_j .

Incremental BC update formulation. Let $BC'_{SCP}(v)$ be the last updated value of the SCP-based BC for a vertex $v \in V$ before the generation of $G_{L+1}(V_{L+1}, E_{L+1})$. For any newly introduced vertex $v \in V_{L+1}$, we initialize $BC'_{SCP}(v) = 0$. Upon the addition of G_{L+1} to the traced TDG, Theorem 4.1 shows that the update from $BC'_{SCP}(v)$ to $BC_{SCP}(v)$ can be expressed as the sum of four incremental terms, denoted by $\lambda(v)$, $\alpha(v)$, $\beta(v)$, and $\gamma(v)$. Each term corresponds to one of the four parts in SCP-based BC decomposition (Figure 8).

Theorem 4.1. The current TDG is $G(V, E) = \bigcup_{k=0}^{L+1} G_k(V_k, E_k)$ with a newly generated subgraph $G_{L+1}(V_{L+1}, E_{L+1})$. The SCP-based BC of any $v \in V$ can be incrementally updated as:

$$BC_{SCP}(v) = BC'_{SCP}(v) + \lambda(v) + \alpha(v) + \beta(v) + \gamma(v), \quad (2)$$

where $BC'_{SCP}(v)$ is the last BC of v before G_{L+1} is generated. The four incremental terms are defined as follows:

$$\lambda(v) = \begin{cases} \sum_{s,t \in V_{L+1}} \frac{\sigma(s, t; v)}{\sigma(s, t)}, & \text{if } v \in V_{L+1}; \\ 0, & \text{otherwise;} \end{cases} \quad (3)$$

and

$$\alpha(v) = \begin{cases} \frac{\sigma(t_{in}^k, t_{out}^k; v)}{\sigma(t_{in}^k, t_{out}^k)} \sum_{V_i \in \Omega_k} N(V_i) \cdot N_{L+1}, & \text{if } v \in V_k; \\ 0, & \text{otherwise;} \end{cases} \quad (4)$$

and

$$\beta(v) = \begin{cases} \eta(v) \cdot \left(\sum_{i=1}^L N(V_i) \right), & \text{if } v \in V_{L+1}; \\ 0, & \text{otherwise;} \end{cases} \quad (5)$$

and

$$\gamma(v) = \begin{cases} \theta(v) \cdot N(V_{L+1}), & \text{if } v \in V_k; \\ 0, & \text{otherwise;} \end{cases} \quad (6)$$

where $1 \leq k \leq L$. $\eta(v) = d(t_{in}^{L+1}; v)$, and $\theta(v) = d'(t_{out}^k; v)$. $d(s; v) = \sum_{t \in V_{L+1}} b(s, t; v)$ and $d'(t; v) = \sum_{s \in V_{L+1}} b(s, t; v)$ are pair-dependencies. $N(V_i)$ is the number of nodes in G_i . Ω_k consists of all the V_i satisfying that there are no edges $(t_{in}^p, t_{in}^q) \in E_0$ for any $p \in [i+1, k]$ and $q \in [k+1, L+1]$.

4.3 Dynamic Tensor Retention

Based on the incremental update formula (2) of SCP-based BC, we propose a dynamic tensor retention algorithm consisting of three main steps, as outlined in Algorithm 1.

Algorithm 1 Dynamic tensor retention algorithm

Input: The current TDG $G(V, E) = \bigcup_{k=0}^{L+1} G_k(V_k, E_k)$ with a new subgraph $G_{L+1}(V_{L+1}, E_{L+1})$, and tensor reservation set R' for the last TDG $G'(V', E') = \bigcup_{k=0}^L G_k(V_k, E_k)$, and $BC'_{SCP}(v)$ for $v \in V'$;
Output: Tensor reservation set R for $G(V, E)$ and $BC_{SCP}(v)$ for $v \in V$;

▼ **Step 1:** Calculate $\lambda(v)$, $\eta(v)$, $\theta(v)$ for each $v \in V_{L+1}$.

- 1: Set $BC_{SCP}(v) = 0$ for $v \in V_{L+1}$, $\text{dist}(s, t) = c(t)$ for $(s, t) \in E_{L+1}$;
- 2: Build a stack S , and an array adjacent (both initially empty);
- 3: **for** $s \in V_{L+1}$ and $s \neq t_{out}^{L+1}$ **do**
- 4: Call `SCP_searching` ($s, V_{L+1}, S, \sigma, \text{dist}, \text{adjacent}, \text{'Forward'}$);
- 5: **while** S is not empty, and pop w from S **do**
- 6: Call `Accumulate` ($s, \emptyset, w, \text{adjacent}, \sigma, d, d', \eta, \theta, \text{'Forward'}$);
- 7: If $w \neq s$, then update $\lambda(w) = \lambda(w) + d(s; w)$;
- 8: **end while**
- 9: **end for**
- 10: Call `SCP_searching` ($t_{out}^{L+1}, V_{L+1}, S, \sigma, \text{dist}, \text{adjacent}, \text{'Backward'}$);
- 11: **while** S is not empty, and pop w from S **do**
- 12: Call `Accumulate` ($\emptyset, t_{out}^{L+1}, w, \text{adjacent}, \sigma, d, d', \eta, \theta, \text{'Backward'}$);
- 13: **end while**
- ▼ **Step 2:** Construct Ω_k for each V_k where $k \in [1, L]$ respectively.
- 14: **for** $i \in [1, k-1]$ and let a flag μ be zero **do**
- 15: **for** $p \in [i+1, k]$ and $q \in [k+1, L+1]$ **do**
- 16: If $(t_{in}^p, t_{in}^q) \in E_0$, then set $\mu = 1$ and break;
- 17: **end for**
- 18: If $\mu == 0$, then put V_i into Ω_k ;
- 19: **end for**
- ▼ **Step 3:** Compute BC_{SCP} of all vertices and build the reservation set R .
- 20: **for** $v \in V_k$ and $k \in [1, L+1]$ **do**
- 21: Calculate $\alpha[v]$, $\beta[v]$, and $\gamma[v]$ based on Eq. (4)-(6);
- 22: Compute $BC_{SCP}(v) = BC'_{SCP}(v) + \lambda(v) + \alpha(v) + \beta(v) + \gamma(v)$;
- 23: **end for**
- 24: Sort all vertices by BC_{SCP} descending, and select top $K\%$ into R ;
- 25: Lock each v in R , and free the lock of vertices in $R' - R$;

Step 1: Calculate $\lambda(v)$, $\eta(v)$, and $\theta(v)$ for each $v \in V_{L+1}$. For any $s, v, t \in V_{L+1}$, $d(s; v)$ and $d'(t; v)$ can be written as

$$d(s; v) = \sum_{\substack{\forall w: (v, w) \in E_{L+1}, \\ \text{dist}(s, w) = \text{dist}(s, v) + c(v)}} \frac{\sigma(s, w)}{\sigma(s, v)} \cdot (1 + d(s; w)), \quad (7)$$

and

$$d'(t; v) = \sum_{\substack{\forall w: (w, v) \in E_{L+1}, \\ \text{dist}(s, v) = \text{dist}(s, w) + c(v)}} \frac{\sigma(v, t)}{\sigma(w, t)} \cdot (1 + d'(t; w)), \quad (8)$$

which indicates that the pair-dependency of s or t on v can be accumulated from the dependencies of its adjacent vertices.

Since $\lambda(v) = \sum_{s \in V_{L+1}} d(s; v)$, Algorithm 1 computes $\lambda(v)$ by iterating over all $s \in V_{L+1}$ and applying Equation (7). For each given $s \in V_{L+1}$, the computation of $d(s; v)$ for all $v \in V_{L+1}$ consists of two phases:

Phase 1: Forward SCP searching (Line 4 in Algorithm 1).

A breadth-first traversal [53] over $G_{L+1}(V_{L+1}, E_{L+1})$ is performed to determine SCP counts $\sigma(s, v)$.

Phase 2: Backward dependency accumulation (Lines 5–8 in Algorithm 1). Vertices are visited in the reverse order of their discovery (i.e., those farthest from s are processed first) to accumulate dependencies according to Equation (7). This yields the pair dependencies $d(s; v)$ and $\eta(v) = d(t_{in}^{L+1}; v)$.

Additionally, a backward SCP searching process from t_{out}^{L+1} is performed to compute $d'(t_{out}^{L+1}; v)$ (Line 10 in Algorithm 1). Based on Equation (8), the vertices are visited in reverse order of discovery again to accumulate pair-dependencies, resulting in $\theta(v) = d'(t_{out}^{L+1}; v)$ (Lines 11–13 in Algorithm 1).

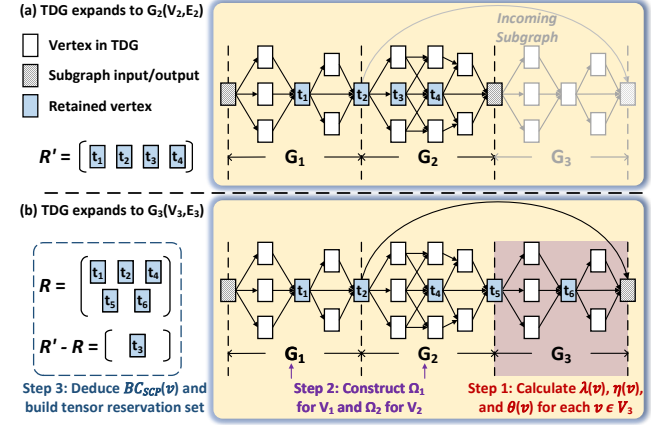


Figure 9. The three main steps of the dynamic tensor retention algorithm, applied to subgraphs G_1 , G_2 , and G_3 .

Step 2: Construct Ω_k for each V_k where $k \in [1, L]$. For each V_i ($i < k$), we check whether there exist any edges $(t_{in}^p, t_{in}^q) \in E_0$ for $p \in [i+1, k]$ and $q \in [k+1, L+1]$. If no such edge exists, we put V_i into Ω_k (Lines 14–19 in Algorithm 1).

Step 3: Deduce $BC_{SCP}(v)$ and build the tensor reservation set. With all parameters in Equations (4)–(6) available, $BC_{SCP}(v)$ for each $v \in V$ can be computed incrementally (Lines 21–22 in Algorithm 1). If $BC_{SCP}(v)$ ranks within the top $K\%$ of all values, v is selected into a reservation set R (as presented in Figure 9), where $K\%$ is dynamically determined based on the current memory usage (Line 24 in Algorithm 1). In T-Control, $K\% = \alpha \frac{\text{residual memory size}}{\text{memory budget}} + 1\%$, where $\alpha \in [10\%, 50\%]$ is empirically determined based on the model size, and the residual memory size is defined as the difference between the memory budget and the currently utilized memory. The adaptive selection of the top $K\%$ achieves tensor retention based on memory usage in runtime. Finally, vertices in R are locked, while previously locked vertices that no longer appear in R (i.e., $R' - R$, as shown in Figure 9) are unlocked and no longer protected.

Remark. Algorithm 1 uses both the traced TDG topology and real-time memory usage to make tensor retention decisions. Its main functions are shown in Appendix A, and its complexity analysis is provided in Appendix B.

5 Segment-centered Memory Management

5.1 Memory Management Principle and Rules

In T-Control, we extend PyTorch’s memory allocator by re-organizing memory around *segments* (contiguous memory

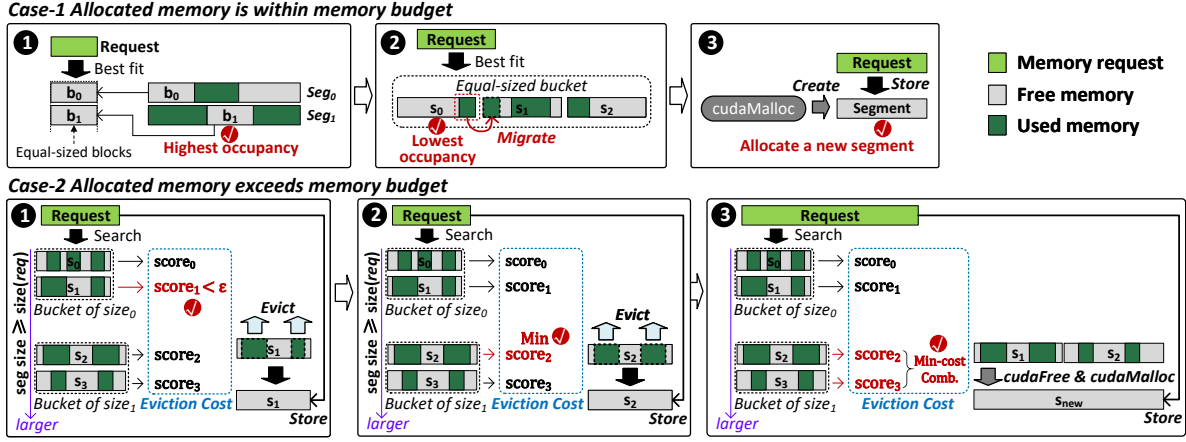


Figure 10. A segment-centered memory management strategy. In Case 2, the search is performed in ascending order of bucket size, efficiently identifying the best-fit solution, as each bucket groups segments of identical size

regions) that are grouped into *buckets* based on their size to reduce allocation search overhead. Each segment maintains metadata such as occupancy and eviction cost, enabling informed decisions for allocation, migration, and eviction. *Blocks* are the fundamental unit of allocation in a segment.

Principle. To achieve the adaptive adjustment of tensor placement in memory (Key idea 2), we propose a segment occupancy polarization principle: *govern memory layout by deliberately concentrating tensor allocations in high-occupancy segments while leaving others minimally used*. By enforcing this polarization, the system encourages a bimodal distribution of memory occupancy—segments are either nearly full or nearly empty—substantially reducing fragmentation and improving predictability in memory availability.

Rules. Based on the design principle, we establish three fine-grained rules to guide fundamental memory operations: *Rule 1: Occupancy-guided best-fit allocation.* Upon receiving an allocation request, the system first searches for free memory blocks that satisfy the request size, following a best-fit policy that prioritizes the smallest suitable blocks. If multiple candidate blocks have the same size, the system selects the one residing in the segment with the highest occupancy, where segment occupancy is quantified as the ratio of occupied size to total size. This rule promotes denser packing of tensors within fewer segments, reducing fragmentation.

Rule 2: Occupancy-guided tensor migration. If an allocation request cannot be fulfilled due to insufficient contiguous free space, the system prioritizes in-place memory compaction before resorting to memory expansion or tensor eviction. It identifies candidate segments whose sizes are large enough to hold the requested tensor and prioritizes those with lower occupancy. Within a selected low-occupancy segment, tensors are progressively migrated to other segments, preferably those with high-occupancy ones (as guided by *Rule 1*). If the

migrations succeed in creating a contiguous block of sufficient size, the new tensor is allocated in the segment.

Rule 3: Cost-aware tensor eviction. This rule reclaims a contiguous block of a required size m_{req} by evicting tensors from an existing segment, while minimizing the eviction cost.

For any tensor t , its recomputation cost is estimated as: $\mathbb{C}_r(t) = c(t) + \sum_{\tau \in \mathcal{N}(t)} c(\tau)$, where $\mathcal{N}(t)$ denotes the set of adjacent tensors, and $c(\cdot)$ is the computation cost of a tensor [24]. Furthermore, we define an *eviction cost score* for a segment with h evictable tensors as:

$$Score(segment) = \sum_{i=1}^h \frac{\mathbb{C}_r(t_i)}{m(t_i) \cdot st(t_i)}, \quad (9)$$

where $m(\cdot)$ and $st(\cdot)$ denote storage size and staleness, respectively. For each segment whose size $\geq m_{req}$, the system selects a subset of evictable tensors that can collectively free up sufficient contiguous space and computes the corresponding eviction cost score as defined in Equation (9). The segment with the lowest *Score* is selected, and corresponding tensors are evicted to fulfill the memory request.

5.2 Memory Management Strategy

Building on the rules described above, we propose a segment-centered memory management strategy (Figure 10). It distinguishes between two cases based on memory availability.

Case 1: The total of the currently used memory and the new request is under the predefined memory budget.

Step 1: If free blocks exist that satisfy the requested size m_{req} , they are collected and sorted in ascending order by size. Select the smallest block that meets the requirement. If multiple blocks share the same size, choose the one in the segment with the highest occupancy, as specified by *Rule 1*. If no suitable block is available, proceed to *Step 2*.

Step 2: Examine segments whose total size is close to m_{req} (typically within the same bucket). Following *Rule 2*, select the segment with the lowest occupancy as a relocation candidate. Iteratively migrate its resident tensors to other blocks using the same placement policy as in *Step 1*. If all required migrations succeed, allocate the requested tensor in the newly vacated segment. Otherwise, proceed to *Step 3*.

Step 3: Allocate a new memory region via `cudaMalloc`.

Case 2: The sum of the currently used memory and the new request exceeds the predefined memory budget.

Step 1: Check whether any segment has a size greater than or equal to m_{req} . If none exists, it proceeds directly to *Step 3*. Otherwise, eligible segments are scanned in ascending size order. Within each segment, tensors are examined sequentially: if a tensor is evictable, the corresponding *Score* in *Rule 3* is accumulated, and its size is added to a running total; if a non-evictable tensor is encountered, the accumulated values are reset for the current segment. This process continues until a contiguous range of evictable tensors is found such that their combined size meets or exceeds m_{req} . If the final *Score* is below a predefined threshold ϵ , the selected tensors are evicted, and the new tensor is allocated in the freed space. If no such segment is found, the process continues to *Step 2*.

Step 2: Among all scanned segments satisfying m_{req} , the one with the lowest eviction cost is selected. Its tensors are evicted to create a contiguous block for allocation. If no segment qualifies, continue to *Step 3*.

Step 3: When no single segment can accommodate m_{req} , we select multiple segments that (i) contain no non-evictable tensors and (ii) incur minimal eviction cost. After evicting all tensors in these segments, a new contiguous region of size m_{req} is allocated.

6 Evaluation

6.1 Experimental Setup

Hardware and environments. We conduct experiments on two hardware platforms: (1) Multi-node experiments are performed on a 64-node cluster, where each node is equipped with 4 NVIDIA A100 GPUs (40 GB each) and two Huawei Kunpeng 920 CPUs (64 cores). Nodes are interconnected via a 4×100 GB/s RoCE high-speed RDMA network. (2) Single-node experiments are executed on a server with 8 NVIDIA A100 GPUs (80 GB each) and two AMD EPYC 7763 CPUs (128 cores), with intra-node communication supported by NVLink (300 GB/s). All experiments are implemented in PyTorch 2.1.0 with CUDA 11.8.

Model and dataset. We adopt representative open-source models sourced from official implementations. All models span two categories: (1) static models, including the GPT [7] and Llama [40] series for language tasks, and ResNet [15], U-Net [35], VGG [37], ViT [46], Inception [39] and DenseNet [19] for vision tasks [18]; and (2) dynamic models, including

AlphaFold [2] for structural biology, as well as LSTM [13], GPT-MoE [11] and BERT4GCN [47] for sequence modeling and graph learning [26]. All models are trained on default datasets from their open-source applications.

Baselines. The evaluation baselines are categorized into four groups: (1) DeepSpeed ZeRO [34] and Zero Bubble [33], representing the state-of-the-art distributed training systems; (2) Megatron-LM [31] and AdaPipe [38], representing the state-of-the-art static TR systems; (3) DTR [24] and DTE [17], representing the state-of-the-art dynamic TR strategies; (4) GMLake [14], representing the state-of-the-art memory defragmentation system for training. Additionally, T-Control* denotes an ablated variant, which disables our memory management strategy to isolate its impact.

6.2 Comparison with Distributed Training Systems

In distributed training, existing systems predominantly adopt either non-recomputation or static tensor rematerialization strategies, as these approaches deliver higher performance compared to dynamic tensor rematerialization (Section 2.2). Hence, the distributed training evaluation compares T-Control against typical non-recomputation and static baselines.

Exp-1: Comparison with non-recomputation systems.

Table 1 compares T-Control with state-of-the-art systems without tensor recomputation. DeepSpeed ZeRO employs ZeRO-DP stage 3 optimization, which equally partitions optimizer states, gradients, and parameters across GPUs to eliminate memory redundancy in data parallelism. However, for GPT and Llama training, this partitioning incurs massive inter-node and intra-node communication, leading to the lowest performance among the baselines. In contrast, Zero Bubble and Megatron-LM support configurable tensor parallelism (TP) and pipeline parallelism (PP), for which we adopt the optimal configurations in Table 1.

Under identical parallelism configurations, T-Control exhibits slightly lower performance than Zero Bubble and Megatron-LM in the memory-sufficient setting, as all methods avoid recomputation while T-Control incurs additional system overhead. However, T-Control further reduces model parallelism (TP or PP) via tensor rematerialization, thereby lowering inter-GPU communication costs and improving overall performance in memory-constrained scenarios.

In the training of GPT 3-121B on 128 GPUs and Llama 3-405B on 256 A100 GPUs, all baseline systems fail due to out-of-memory (OOM) errors. By contrast, T-Control utilizes tensor recomputation to mitigate memory bottlenecks, enabling successful training and highlighting its strong suitability for large-scale models under tight memory budgets.

Exp-2: Comparison with static TR systems. Table 2 compares T-Control with existing static TR systems. Despite

Table 1. Execution time (s/iteration) of T-Control and non-recomputation baselines. DeepSpeed ZeRO uses ZeRO-DP stage 3 optimization for its parallelism configuration. DeepSpeed Zero and Zero Bubble lack support for both static and dynamic tensor rematerialization (TR), and TR is disabled in Megatron-LM for consistency. DP, TP, PP, and MBS represent Data Parallel size, Tensor Parallel size, Pipeline Parallel size, and Micro-Batch Size, respectively. Llama 3-405B is an extremely large model.

Model	DeepSpeed Zero	Zero Bubble	Megatron-LM (w/o TR)	T-Control (w/o TR)	T-Control (with TR)
Parallelism config. GPT 3-1.5B (32 GPUs)	Stage=3,MBS=4 21.54s	DP=16,TP=1,PP=2,MBS=4 5.78s	DP=16,TP=1,PP=2,MBS=4 6.33s	DP=16,TP=1,PP=2,MBS=4 6.41s	DP=32,TP=1,PP=1,MB=4 5.61s
Parallelism config. Llama 2-7B (32 GPUs)	Stage=3,MBS=2 26.63s	DP=4,TP=1,PP=8,MBS=2 8.46s	DP=4,TP=1,PP=8,MBS=2 9.22s	DP=4,TP=1,PP=8,MBS=2 9.31s	DP=8,TP=1,PP=4,MBS=2 8.02s
Parallelism config. GPT 3-121B (128 GPUs)	Stage=3,MBS=1 out-of-memory	DP=1,TP=4,PP=32,MBS=1 out-of-memory	DP=1,TP=4,PP=32,MBS=1 out-of-memory	DP=1,TP=4,PP=32,MBS=1 out-of-memory	DP=1,TP=4,PP=32,MBS=1 91.59s
Parallelism config. Llama 3-405B (256 GPUs)	Stage=3,MBS=1 out-of-memory	DP=1,TP=4,PP=64,MBS=1 out-of-memory	DP=1,TP=4,PP=64,MBS=1 out-of-memory	DP=1,TP=4,PP=64,MBS=1 out-of-memory	DP=1,TP=4,PP=64,MBS=1 74.31s

incurring runtime decision-making overhead, T-Control outperforms Megatron-ML and AdaPipe, making it the first dynamic rematerialization system to exceed the performance of static methods. The main reasons are as follows.

Existing static TR systems mainly optimize tensor retention and eviction schedules but lack fine-grained control over tensor placement and lifetimes, as determining the optimal memory locations and lifetimes for all tensors before training requires solving a global optimization problem, which is intractable due to the vast scheduling space and complex tensor dependencies. This limitation yields memory fragmentation and inefficient memory usage. When contiguous free memory is insufficient, the systems must evict in-memory tensors and later recompute them, incurring additional overhead.

In contrast, T-Control integrates rematerialization scheduling with memory management at runtime, allowing decisions to adapt to the current memory state. By proactively migrating tensors and coalescing fragmented memory into larger contiguous segments, T-Control enables more successful tensor allocations without triggering eviction and recomputation. The significant reduction in recomputation count directly contributes to its performance gains (Table 2).

Table 2. The average time and tensor recomputation count (TRC) per iteration of different systems in the training of various models.

Model and Parallelism config.	Megatron-LM		AdaPipe		T-Control	
	Time	TRC	Time	TRC	Time	TRC
GPT 3-1.7B (8 GPUs) DP=4,TP=1,PP=2,MBS=4	18.14s	8,192	20.86s	8,173	16.51s	4,392
GPT 3-7.5B (32 GPUs) DP=8,TP=1,PP=4,MBS=1	22.58s	68,095	26.05s	56,339	18.08s	35,314
Llama 2-13B (32 GPUs) DP=2,TP=2,PP=8,MBS=2	34.02s	170,238	31.07s	119,855	30.02s	101,843
Llama 2-70B (128 GPUs) DP=2,TP=4,PP=16,MBS=1	71.52s	173,624	63.18s	120,348	60.28s	104,634

Exp-3: Overall performance. Figure 11 presents the performance evaluation using the optimal parallelism configuration for each system on 256 GPUs. T-Control achieves the

throughput improvements of $1.05\times\sim 1.58\times$ for GPT 3 models and $1.10\times\sim 1.22\times$ for Llama 2 models compared to DeepSpeed ZeRO, Megatron-LM, Zero Bubble, and AdaPipe. Both DeepSpeed ZeRO and Zero Bubble encounter OOM errors as the model size increases, as they are non-recomputation systems. We summarize three key observations as follows:

First, compared to Zero Bubble, the performance advantage of T-Control grows as the number of GPUs increases. T-Control leverages TR to fit more computation tasks within each GPU’s memory. As a result, it reduces TP and PP and allows a higher DP, thereby lowering the frequency and volume of inter-GPU communication.

Second, as the model size grows, T-Control demonstrates an increasing performance advantage over Megatron-LM. Specifically, when training GPT 3-7.5B, T-Control achieves a geometric mean speedup of $1.25\times$ over Megatron-LM, and the speedup increases to $1.33\times$ when training GPT 3-121B. Although T-Control incurs runtime overhead, it dynamically retains tensors based on real-time memory availability, significantly reducing both tensor eviction and rematerialization (Table 2). In contrast, Megatron-LM’s static decisions are fixed before training, lacking adaptability.

Third, T-Control achieves a high throughput comparable to AdaPipe. While AdaPipe is specialized for transformer models, e.g., GPT and Llama series, T-Control matches or exceeds its performance. This is because T-Control integrates rematerialization scheduling with memory management at runtime, while AdaPipe focuses solely on the rematerialization scheduling design before training.

6.3 Comparison with Dynamic Methods

Current dynamic TR methods only support model training on a single node with multiple GPUs. Therefore, this evaluation is conducted on a server equipped with 8 NVIDIA A100 GPUs. Moreover, both static and dynamic models are tested, since dynamic TR can be applied to their training processes.

Exp-4: Evaluation with static and dynamic models. Table 3 presents the evaluation on static and dynamic models

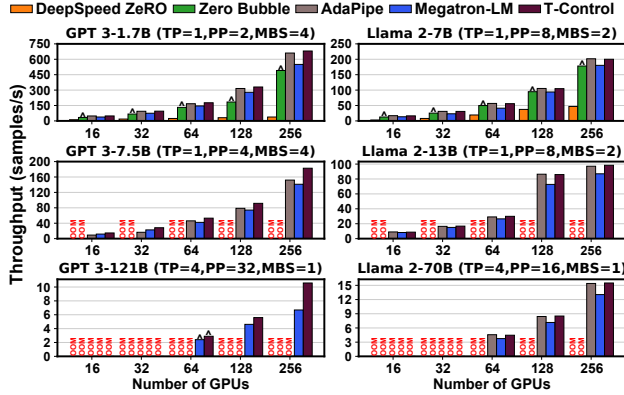


Figure 11. Throughput of GPT-3 and Llama-2 training. The marker $\wedge$ indicates the optimal parallelism configurations: (TP=1, PP=4, MBS=2) for GPT 3-1.7B, (TP=1, PP=8, MBS=2) for Llama 2-7B, and (TP=4, PP=16, MBS=1) for GPT 3-121B. DP is determined by GPU count and other parallelism settings.

with various architectures. Compared with the state-of-the-art dynamic methods, T-Control achieves a geometric mean speedup of $1.17\times$ over DTR, $1.25\times$ over DTE, and $1.47\times$ over GMLake (with DTR). These performance gains stem from two primary sources. First, GMLake incurs substantial overhead due to frequent and time-consuming calls to virtual memory APIs. However, T-Control follows three new rules for memory management without using virtual memory APIs. Second, DTR and DTE evict tensors solely based on the cost model, which may result in the eviction of hub tensors in the TDG and deep recursive rematerialization, significantly increasing overhead. In contrast, T-Control leverages SCP-based BC values to identify and retain structurally important tensors in the TDG, thereby avoiding deep recursive calls.

Table 3. The performance (ms/iter.) of dynamic tensor rematerialization methods for the training of static and dynamic models with various architectures on 8 NVIDIA A100 GPUs.

Static model	DTR	DTE	GMLake+DTR	T-Control
ResNet-50(MBS=56)	207.7	208.3	495.8	197.1
U-Net(MBS=32)	90.1	92.5	97.9	89.5
VGG-16(MBS=32)	2.7	2.8	3.1	2.6
Dynamic model	DTR	DTE	GMLake+DTR	T-Control
AlphaFold-2(MBS=10)	7641.2	9891.6	8598.7	5014.2
LSTM(MBS=256)	1062.9	1147.6	1139.4	746.7
BERT4GCN(MBS=16)	1874.4	1937.8	2091.5	1774.5

Exp-5: Evaluation with different memory budgets. Figure 12 shows the per-step training overhead of various dynamic methods under different memory budget ratios. T-Control outperforms the state-of-the-art dynamic strategies, achieving a speedup of $1.04\times\sim 1.74\times$ over DTR, $1.09\times\sim 1.91\times$ over DTE, and $1.97\times\sim 3.55\times$ over GMLake (with DTR). As

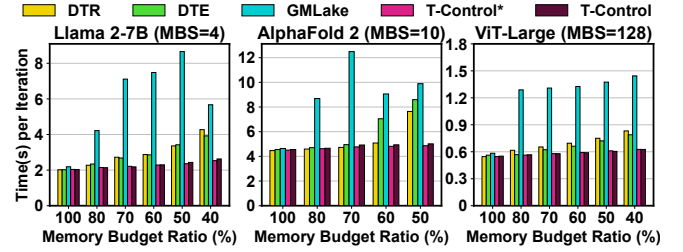


Figure 12. Comparison between different dynamic methods using data parallelism on 8 NVIDIA A100 GPUs.

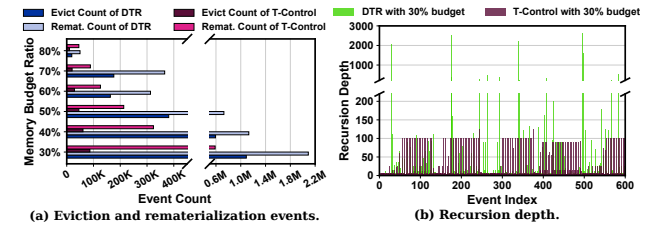


Figure 13. Tensor eviction and rematerialization counts per training step, and the recursion depth of each TR in different methods for Llama 2-7B training on 8 A100 GPUs.

the memory budget ratio decreases, the performance gains become more pronounced, primarily due to the greater reduction in eviction and rematerialization events (Figure 13).

Compared with T-Control*, T-Control enables the segment-centered memory management strategy. This strategy reduces memory fragmentation (Figure 14), preventing unnecessary tensor evictions and recomputations. While enabling memory management introduces nontrivial overheads, the performance gains offset this cost. Consequently, T-Control and T-Control* exhibit comparable performance (Figure 12).

6.4 Performance Analysis

Exp-6: Eviction and rematerialization counts. As shown in Figure 13(a), T-Control reduces eviction events by up to 92% and rematerialization events by up to 71%, compared to DTR. The improvement is attributed to two key factors. First, our retention strategy prioritizes retaining tensors based on their dependency in the TDG, rather than solely relying on the cost model. Second, the dynamic memory management strategy enhances memory efficiency by preserving low-occupancy memory segments for future allocations and actively migrating to form contiguous memory regions.

Exp-7: Recursion depth. Figure 13(b) further shows that T-Control significantly reduces the recursion depth of each TR event. The maximum recursion depth under T-Control is 126, which is $21\times$ lower than that of DTR. This substantial reduction is attributed to Algorithm 1, which selectively retains tensors with high SCP-based BC values, thereby effectively preventing deep recursive recomputation.

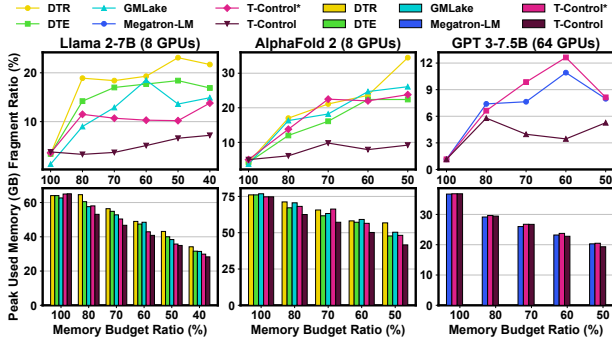


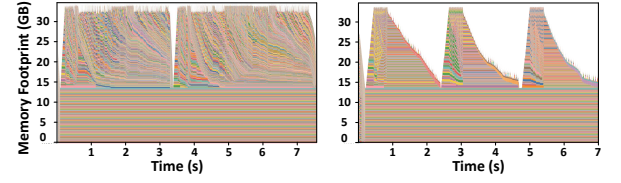
Figure 14. The fragment ratio and peak memory usage of different methods with various memory budgets.

Exp-8: Fragmentation ratio and peak memory usage. As shown in Figure 14, T-Control achieves the lowest fragmentation ratio and peak memory usage under various memory budgets, compared with dynamic methods including DTR, DTE, and GMLake. Furthermore, we observe two phenomena. First, GMLake leverages virtual memory to reduce fragmentation and improve utilization, resulting in efficiency comparable to T-Control*. This demonstrates that both minimizing eviction/rematerialization and reducing fragmentation via virtual memory are effective in improving memory efficiency. Second, compared to T-Control*, T-Control further reduces both memory fragmentation and peak usage, as the three proposed rules enable more efficient memory management, although with a slight running time overhead (Figure 12).

In distributed training, T-Control consistently maintains a fragmentation ratio below 6%, while Megatron-LM (static TR system) exhibits up to 11% (Figure 14). Although T-Control occasionally incurs slightly higher peak usage than Megatron-LM due to the overhead of fine-grained memory tracking, it achieves a more favorable trade-off between memory efficiency and runtime performance (Figure 11).

Exp-9: GPU-memory footprint. As shown in Figure 15, under a 35GB memory budget, DTR enables the single batch fine-tuning of Llama 2-7B Lora, but results in an irregular memory accumulation, which is due to massive fragments and multiple memory allocations required by each deep recursion. This irregular memory footprint reduces the training efficiency. However, by effectively avoiding deep recursion and memory fragments, T-Control achieves a regular memory footprint, which leads to over 1.5 $\times$ speedup in the training performance.

Exp-10: Dynamic tensor retaining results. Figure 16 shows that both the computation-from-scratch and the incremental algorithm (Algorithm 1) yield identical SCP-based BC values for all vertices in the TDG, thereby validating the correctness of Algorithm 1. The red-colored vertices in the figure, which possess the highest SCP-based BC values, form a shortest path from the first to the last vertex in the TDG.



(a) DTR memory footprint. (b) T-Control memory footprint.

Figure 15. GPU-memory footprint of DTR and T-Control for a single-batch fine-tuning of Llama 2-7B Lora with 35GB memory budget on one NVIDIA A100 GPU.

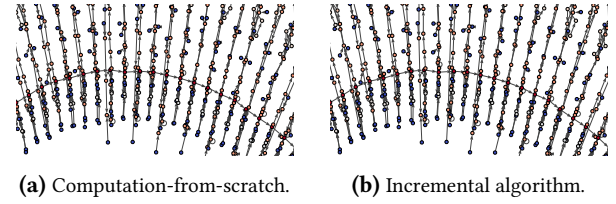


Figure 16. SCP-based BC of each vertex in the TDG of GPT 3-7.5B training based on different methods computing BC. The vertex color from blue to red means the increasing value. The red vertices are those retained by Algorithm 1, forming a shortest path from the first vertex to the last in the TDG.

Table 4. The per-iteration system overhead of T-Control in model training. Llama 3-405B represents an extremely large-scale static model that requires 256 A100 GPUs (40 GB each) to train successfully, whereas existing dynamic models typically can run on 8 A100 GPUs (80 GB each).

Model type	Model	GPU num.	System cost	Time per iter.	Percent.
Static	GPT-3 7.5B	8	1.786s	45.4s	3.96%
	Llama 2-7B	8	0.074s	2.2s	3.36%
Dynamic	AlphaFold-2	8	0.361s	7.3s	4.95%
	LSTM	8	0.051s	1.13s	4.51%
Large	GPT 3-121B	256	2.103s	48.7s	4.32%
	Llama 3-405B	256	3.402s	74.3s	4.58%

Notably, any path from subgraph G_i to G_j must traverse at least one of these red vertices. This observation confirms that the SCP-based BC metric effectively quantifies vertex importance with respect to the TDG topology. By identifying these critical vertices, Algorithm 1 could exploit limited memory to keep more useful data.

Exp-11: System overhead. T-Control’s system overhead refers to the execution time required for the tensor retention (Algorithm 1, including three steps) and the segment-centered memory management (Figure 10, including allocation, eviction, migration). Table 4 reports the runtime cost introduced by T-Control when training different models. In all cases, the system overhead remains below 5% of the total execution time. This is due to two reasons. First, although Algorithm 1 involves non-trivial computational complexity,

Table 5. Execution time (s/iteration) of Megatron-LM and T-Control for the training of GPT-MoE models.

Model	Parallelism Configuration	Megatron-LM	T-Control
GPT-MoE 1.7B	DP=1, TP=1, PP=4, MBS=4, 4 GPUs	21.91	19.97
GPT-MoE 1.7B	DP=2, TP=1, PP=4, MBS=4, 8 GPUs	11.42	10.37
GPT-MoE 3.6B	DP=4, TP=1, PP=4, MBS=4, 16 GPUs	14.09	13.62
GPT-MoE 3.6B	DP=8, TP=1, PP=4, MBS=4, 32 GPUs	8.31	7.91
GPT-MoE 10B	DP=8, TP=1, PP=8, MBS=1, 64 GPUs	9.76	9.14
GPT-MoE 10B	DP=16, TP=1, PP=8, MBS=1, 128 GPUs	6.19	5.95

Table 6. Execution time (s/iteration) of DTR and T-Control for training different models on 8 NVIDIA A100 GPUs under varying memory budgets.

Model	System	Memory Budget				
		80%	70%	60%	50%	40%
Inception-V4	DTR	2.05	2.11	2.15	2.22	2.25
	T-Control	1.45	1.49	1.53	1.61	1.65
DenseNet-121	DTR	1.75	1.83	1.86	1.95	2.02
	T-Control	1.52	1.61	1.64	1.71	1.73
ResNet-110	DTR	1.34	1.38	1.40	1.43	1.48
	T-Control	1.14	1.18	1.21	1.25	1.27

particularly in Step 1 and Step 3 (Appendix B), it is executed only once per layer. For example, Llama 2-7B consists of merely 32 layers, so the cumulative cost remains small. Second, the overheads of allocation and eviction are minimal and are largely hidden by GPU multi-stream execution. Tensor migration is more expensive per invocation, but it occurs rarely enough that its total contribution is negligible.

Although dynamic models and larger static models inevitably involve more memory operations and thus slightly higher costs than small static models, the efficient design of T-Control ensures that the additional overhead remains minimal. Such a lightweight cost is well justified in practice, as the throughput improvements consistently outweigh it, enabling T-Control to achieve performance on par with state-of-the-art static TR methods (Section 6.2) and to surpass existing dynamic TR strategies (Section 6.3).

Exp-12: Performance on different model architectures.

Although parallel branches and skip connections may cause BC values to spread evenly, T-Control handles these cases effectively. For MoE models, the runtime-traced TDG includes only executed branches, so non-executed branches are not traced and do not dilute BC scores. For skip connections, the performance improvement on ResNet-50 and U-Net training (Table 3) demonstrates that T-Control retains critical tensors.

As shown in Tables 5 and 6, T-Control outperforms state-of-the-art systems across diverse model architectures. In general, T-Control preserves tensors with higher BC-values, such as layer or branch inputs and outputs. In rare cases where multiple important tensors have identical BC scores, any of them may be retained, which would not mislead the retention decisions.

7 Related Works

Static tensor rematerialization. Sublinear [10] is one of the earliest static approaches, demonstrating that an N -layer feedforward network can be trained with $O(\sqrt{N})$ memory and $O(N \log N)$ recomputation cost. Checkmate [20] formulates rematerialization as an integer linear program to derive globally optimal eviction schedules under strict memory budgets. Magis [9] incorporates TASO’s graph transformation [21] to enable computational graph optimization within rematerialization. Capuchin [32] focuses on coordinating tensor swapping and recomputation. However, it assumes iteration-level stability in tensor access patterns and therefore cannot effectively support dynamic workflows, such as the training processes of AlphaFold and MoE models. Megatron-LM [25, 31] applies static recomputation to reduce peak memory usage, partially mitigating runtime fragmentation. H3T [45] treats rematerialization and other optimizations as a joint decision space for integrated strategy formulation. Adapipe [38] specializes rematerialization plans for Transformer models [41]. More recently, GMLake [14] proposes VMS, consolidating fragmented memory into logically contiguous blocks for data allocation, significantly reducing memory fragmentation.

Dynamic tensor rematerialization. To the best of our knowledge, DTR [9] remains the most widely adopted dynamic rematerialization method, using heuristics to guide eviction decisions during execution. However, dynamic approaches often suffer from fragmentation, which hampers memory reuse and leads to allocation failures. To address this, MegTaichi [17] introduces DTE, incorporating fragmentation awareness into eviction scheduling. Coop [50] further enhances dynamic rematerialization with a fragmentation-aware sliding-window policy integrated with the OneFlow allocator [49], albeit without distributed training support. Nevertheless, existing dynamic methods still rely on local heuristics and overlook long-range tensor dependencies, resulting in sub-optimal eviction decisions.

8 Conclusion

This work presents an in-depth analysis of dynamic tensor rematerialization methods. Based on two key ideas, we develop T-Control, a system for DNN training, which efficiently integrates dynamic tensor retention with segment-centered memory management. Experimental results demonstrate T-Control’s high performance and effectiveness.

Acknowledgments

We thank all reviewers for their valuable feedback. We also thank Keval Vora for his suggestions on improving the paper. This work is supported in part by the National Natural Science Foundation of China, under grant numbers 62032023, T2125013, and 62172391.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2024. Gpt-4 technical report. (2024). arXiv:2303.08774 [cs.CL] doi:10.48550/arXiv.2303.08774
- [2] Gustaf Ahndritz, Nazim Bouatta, Christina Floristean, Sachin Kadyan, Qinghui Xia, William Gerecke, Timothy J O'Donnell, Daniel Berenberg, Ian Fisk, Niccolò Zanichelli, et al. 2024. OpenFold: Retraining AlphaFold2 yields new insights into its learning mechanisms and capacity for generalization. *Nature Methods* (2024), 1–11. doi:10.1038/s41592-024-02272-z
- [3] Alex Bavelas. 1950. Communication patterns in task-oriented groups. *Journal of the acoustical society of America* 22, 6 (1950), 725–730. doi:10.1121/1.1906679
- [4] Olivier Beaumont, Lionel Eyraud-Dubois, and Alena Shilova. 2021. Efficient Combination of Rematerialization and Offloading for Training DNNs. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 23844–23857. https://proceedings.neurips.cc/paper_files/paper/2021/file/c8461bf13fca8a2b9912ab2eb1668e4b-Paper.pdf
- [5] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of mathematical sociology* 25, 2 (2001), 163–177. doi:10.1080/0022250X.2001.9990249
- [6] Ulrik Brandes. 2008. On variants of shortest-path betweenness centrality and their generic computation. *Social Networks* 30, 2 (2008), 136–145. doi:10.1016/j.socnet.2007.11.001
- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6fbcb4967418bfb8ac142f64a-Paper.pdf
- [8] Daniele Castellana and Davide Bacciu. 2022. A tensor framework for learning in structured domains. *Neurocomputing* 470 (2022), 405–426. doi:10.1016/j.neucom.2021.05.110
- [9] Renze Chen, Zijian Ding, Size Zheng, Chengrui Zhang, Jingwen Leng, Xuanzhe Liu, and Yun Liang. 2024. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 607–621. doi:10.1145/3620666.3651330
- [10] Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. arXiv:1604.06174 [cs.LG] doi:10.48550/arXiv.1604.06174
- [11] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39. <http://jmlr.org/papers/v23/21-0998.html>
- [12] Linton C Freeman. 1977. A set of measures of centrality based on betweenness. *Sociometry* (1977), 35–41. doi:10.2307/3033543
- [13] Alex Graves. 2012. *Long Short-Term Memory*. Springer Berlin Heidelberg, Berlin, Heidelberg, 37–45. doi:10.1007/978-3-642-24797-2_4
- [14] Cong Guo, Rui Zhang, Jiale Xu, Jingwen Leng, Zihan Liu, Ziyu Huang, Minyi Guo, Hao Wu, Shouren Zhao, Junping Zhao, and Ke Zhang. 2024. GMLake: Efficient and Transparent GPU Memory Defragmentation for Large-scale DNN Training with Virtual Memory Stitching. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 450–466. doi:10.1145/3620665.3640423
- [15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778. https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper
- [16] Horst Herrlich. 1974. Topological structures. *Math. Centre Tracts* 52 (1974), 59–122. <https://www.mathunion.org/fileadmin/ICM/Proceedings/ICM1974.2/ICM1974.2.ocr.pdf>
- [17] Zhongzhe Hu, Junmin Xiao, Zheyue Deng, Mingyi Li, Kewei Zhang, Xiaoyang Zhang, Ke Meng, Ninghui Sun, and Guangming Tan. 2022. MegTaiChi: dynamic tensor-based memory management optimization for DNN training. In *Proceedings of the 36th ACM International Conference on Supercomputing* (Virtual Event) (ICS '22). Association for Computing Machinery, New York, NY, USA, Article 25, 13 pages. doi:10.1145/3524059.3532394
- [18] Zhongzhe Hu, Junmin Xiao, Ninghui Sun, and Guangming Tan. 2022. Fast and accurate variable batch size convolution neural network training on large scale distributed systems. *Concurrency and Computation: Practice and Experience* 34, 21 (2022), e7119. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.7119 doi:10.1002/cpe.7119
- [19] Forrest Iandola, Matt Moskewicz, Sergey Karayev, Ross Girshick, Trevor Darrell, and Kurt Keutzer. 2014. DenseNet: Implementing Efficient ConvNet Descriptor Pyramids. arXiv:1404.1869 [cs.CV] doi:10.48550/arXiv.1404.1869
- [20] Paras Jain, Ajay Jain, Aniruddha Nrusimha, Amir Gholami, Pieter Abbeel, Joseph Gonzalez, Kurt Keutzer, and Ion Stoica. 2020. Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization. In *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze (Eds.), Vol. 2. 497–511. https://proceedings.mlsys.org/paper_files/paper/2020/file/0b816ae8f06f8dd3543dc3d9ef196cab-Paper.pdf
- [21] Zhihao Jia, Oded Padon, James Thomas, Todd Warszawski, Matei Zaharia, and Alex Aiken. 2019. TASO: optimizing deep learning computation with automatic generation of graph substitutions. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles* (Huntsville, Ontario, Canada) (SOSP '19). Association for Computing Machinery, New York, NY, USA, 47–62. doi:10.1145/3341301.3359630
- [22] Zhigang Jin, Manyue Tao, Xiaofang Zhao, and Yi Hu. 2022. Social media sentiment analysis based on dependency graph and co-occurrence graph. *Cognitive Computation* 14, 3 (2022), 1039–1054. doi:10.1007/s12559-022-10004-8
- [23] Mark S. Johnstone and Paul R. Wilson. 1998. The memory fragmentation problem: solved?. In *Proceedings of the 1st International Symposium on Memory Management* (Vancouver, British Columbia, Canada) (ISMM '98). Association for Computing Machinery, New York, NY, USA, 26–36. doi:10.1145/286860.286864
- [24] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. 2021. Dynamic Tensor Rematerialization. arXiv:2006.09616 [cs.LG] doi:10.48550/arXiv.2006.09616
- [25] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of Machine Learning and Systems*, D. Song, M. Carbin, and T. Chen (Eds.), Vol. 5. Curran, 341–353. https://proceedings.mlsys.org/paper_files/paper/2023/file/80083951326cf5b35e5100260d64ed81-Paper-mlsys2023.pdf

- [26] Mingyi Li, Junmin Xiao, Kewei Zhang, Zhiheng Lin, Chaoyang Shui, Ke Meng, Zehua Wang, Yunfei Pang, and Guangming Tan. 2024. A Coordinated Strategy for GNN Combining Computational Graph and Operator Optimizations. In *Proceedings of the 38th ACM International Conference on Supercomputing* (Kyoto, Japan) (ICS '24). Association for Computing Machinery, New York, NY, USA, 460–472. doi:10.1145/3650200.3661896
- [27] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, and Soumith Chintala. 2020. PyTorch Distributed: Experiences on Accelerating Data Parallel Training. arXiv:2006.15704 [cs.DC] <https://arxiv.org/abs/2006.15704>
- [28] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2025. Deepseek-v3 technical report. (2025). arXiv:2412.19437 [cs.CL] doi:10.48550/arXiv.2412.19437
- [29] Mugilan Mariappan, Joanna Che, and Keval Vora. 2021. DZiG: sparsity-aware incremental processing of streaming graphs. In *Proceedings of the Sixteenth European Conference on Computer Systems* (Online Event, United Kingdom) (EuroSys '21). Association for Computing Machinery, New York, NY, USA, 83–98. doi:10.1145/3447786.3456230
- [30] Mugilan Mariappan and Keval Vora. 2019. GraphBolt: Dependency-Driven Synchronous Processing of Streaming Graphs. In *Proceedings of the Fourteenth EuroSys Conference 2019* (Dresden, Germany) (EuroSys '19). Association for Computing Machinery, New York, NY, USA, Article 25, 16 pages. doi:10.1145/3302424.3303974
- [31] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prithvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient large-scale language model training on GPU clusters using megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis* (St. Louis, Missouri) (SC '21). Association for Computing Machinery, New York, NY, USA, Article 58, 15 pages. doi:10.1145/3458817.3476209
- [32] Xuan Peng, Xuanhua Shi, Hulin Dai, Hai Jin, Weiliang Ma, Qian Xiong, Fan Yang, and Xuehai Qian. 2020. Capuchin: Tensor-based GPU Memory Management for Deep Learning. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) (ASPLOS '20). Association for Computing Machinery, New York, NY, USA, 891–905. doi:10.1145/3373376.3378505
- [33] Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. 2024. Zero Bubble (Almost) Pipeline Parallelism. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=tuzTN0eIO5>
- [34] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory optimizations Toward Training Trillion Parameter Models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–16. doi:10.1109/SC41405.2020.00024
- [35] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Nassir Navab, Joachim Hornegger, William M. Wells, and Alejandro F. Frangi (Eds.). Springer International Publishing, Cham, 234–241. doi:10.1007/978-3-319-24574-4_28
- [36] Gert Sabidussi. 1966. The Centrality Index of a Graph. *Psychometrika* 31, 4 (1966), 581–603. doi:10.1007/BF02289527
- [37] Karen Simonyan and Andrew Zisserman. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556 [cs.CV] doi:10.48550/arXiv.1409.1556
- [38] Zhenbo Sun, Huanqi Cao, Yuanwei Wang, Guanyu Feng, Shengqi Chen, Haojie Wang, and Wenguang Chen. 2024. AdaPipe: Optimizing Pipeline Parallelism with Adaptive Recomputation and Partitioning. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) (ASPLOS '24). Association for Computing Machinery, New York, NY, USA, 86–100. doi:10.1145/3620666.3651359
- [39] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. 2016. Rethinking the Inception Architecture for Computer Vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. https://www.cv-foundation.org/openaccess/content_cvpr_2016/html/Szegedy_Rethinking_the_Inception_CVPR_2016_paper.html
- [40] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] doi:10.48550/arXiv.2302.13971
- [41] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems* (Long Beach, California, USA) (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010. <https://dl.acm.org/doi/10.5555/3295222.3295349>
- [42] Keval Vora, Rajiv Gupta, and Guoqing Xu. 2016. Synergistic Analysis of Evolving Graphs. *ACM Trans. Archit. Code Optim.* 13, 4, Article 32 (Oct. 2016), 27 pages. doi:10.1145/2992784
- [43] Keval Vora, Rajiv Gupta, and Guoqing Xu. 2017. KickStarter: Fast and Accurate Computations on Streaming Graphs via Trimmed Approximations. In *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems* (Xi'an, China) (ASPLOS '17). Association for Computing Machinery, New York, NY, USA, 237–251. doi:10.1145/3037697.3037748
- [44] Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. 2018. Non-local Neural Networks. In *Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 7794–7803. doi:10.1109/CVPR.2018.00813
- [45] Yuzhong Wang, Xu Han, Weilin Zhao, Guoyang Zeng, Zhiyuan Liu, and Maosong Sun. 2023. H3T: Efficient Integration of Memory Optimization and Parallelism for Large-scale Transformer Training. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 38311–38334. https://proceedings.neurips.cc/paper_files/paper/2023/file/7886b89aced4d37dd25a6f32854bf3f9-Paper-Conference.pdf
- [46] Bichen Wu, Chenfeng Xu, Xiaoliang Dai, Alvin Wan, Peizhao Zhang, Zhicheng Yan, Masayoshi Tomizuka, Joseph Gonzalez, Kurt Keutzer, and Peter Vajda. 2020. Visual Transformers: Token-based Image Representation and Processing for Computer Vision. arXiv:2006.03677 [cs.CV] doi:10.48550/arXiv.2006.03677
- [47] Zeguan Xiao, Jiarun Wu, Qingliang Chen, and Congjian Deng. 2021. BERT4GCN: Using BERT Intermediate Layers to Augment GCN for Aspect-based Sentiment Classification. arXiv:2110.00171 [cs.CL] doi:10.48550/arXiv.2110.00171
- [48] Mengwei Xu, Wangsong Yin, Dongqi Cai, Rongjie Yi, Daliang Xu, Qipeng Wang, Bingyang Wu, Yihao Zhao, Chen Yang, Shihe Wang, Qiyang Zhang, Zhenyan Lu, Li Zhang, Shangguang Wang, Yuanchun Li, Yunxin Liu, Xin Jin, and Xuanzhe Liu. 2024. A Survey of Resource-efficient LLM and Multimodal Foundation Models. arXiv:2401.08092 [cs.LG] doi:10.48550/arXiv.2401.08092
- [49] Jinhui Yuan, Xinqi Li, Cheng Cheng, Juncheng Liu, Ran Guo, Shenghang Cai, Chi Yao, Fei Yang, Xiaodong Yi, Chuan Wu, Haoran Zhang, and Jie Zhao. 2022. OneFlow: Redesign the Distributed Deep Learning Framework from Scratch. arXiv:2110.15032 [cs.DC] doi:10.48550/arXiv.2110.15032

- [50] Jianhao Zhang, Shihan Ma, Peihong Liu, and Jinhui Yuan. 2023. Coop: Memory is not a Commodity. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 49870–49882. https://proceedings.neurips.cc/paper_files/paper/2023/file/9c534edc7ac1d6438216311be6d4eb2-Paper-Conference.pdf
- [51] Xiaoyang Zhang, Junmin Xiao, and Guangming Tan. 2021. I/O lower bounds for auto-tuning of convolutions in CNNs. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (Virtual Event, Republic of Korea) (PPoPP '21). Association for Computing Machinery, New York, NY, USA, 247–261. doi:10.1145/3437801.3441609
- [52] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. 2025. A Survey of Large Language Models. arXiv:2303.18223 [cs.CL] <https://arxiv.org/abs/2303.18223>
- [53] Rong Zhou and Eric A. Hansen. 2006. Breadth-first heuristic search. *Artificial Intelligence* 170, 4 (2006), 385–408. doi:10.1016/j.artint.2005.12.002

A Main Functions in Algorithm 1

The three functions, i.e., SCP_searching, Search_usingBFT, and Accumulate, enable the dynamic calculation of SCP-based BCs for all vertices in the traced TDG (Algorithm 1).

First, SCP_searching function initiates a breadth-first traversal (BFT) from a source vertex, managing the exploration in either forward or backward directions to identify shortest paths and update distances, path counts, and adjacency lists (Algorithm 2).

Second, Search_usingBFT function supports BFT search by handling individual vertex explorations, updating shortest path distances and counts based on edge directions, and maintaining adjacency relationships (Algorithm 3).

Third, Accumulate function computes dependency scores for vertices based on shortest paths, updating centrality metrics using forward and backward dependency calculations, which are crucial for determining a vertex's influence in the TDG's connectivity (Algorithm 4).

Together, these functions efficiently compute SCP-based BCs by tracking shortest paths and their contributions to vertex importance.

B Time/Space complexity of Algorithm 1

As discussed in Section 4, the TDG on each GPU is denoted as $G(V, E) = \bigcup_{k=0}^L G_k(V_k, E_k)$.

Time complexity: The time complexity of Algorithm 1 is

$$O\left(\sum_{k=1}^L |V_k|(|E_k| + |V_k| \log(|V_k|)) + L^2 + |V|L\right). \quad (10)$$

Specifically, (1) at Step 1, the time complexity of BFT on G_k is $O(|V_k|(|E_k| + |V_k| \log(|V_k|)))$ in SCP searching process, and the accumulation process owns the time complexity $O(|V_k|(|E_k| + |V_k|))$. (2) At Step 2, the time complexity of constructing all Ω_k is $O(L^2)$. (3) Step 3 involves the

Algorithm 2 SCP_searching Function

Input: $s, V_k, S, \sigma(*, *)$, $\text{dist}(*, *)$, $\text{adjacent}(*, *)$, Direction;

Output: Updated $S, \sigma(*, *)$, $\text{dist}(*, *)$, $\text{adjacent}(*, *)$;

▼ **Initialization.**

```

1: for  $w \in V_k$  do
2:   Let  $\text{adjacent}(w) = \text{empty list}$ ,  $\text{dist}(s, w) = \infty$ ,  $\sigma(s, w) = 0$ ;
3: end for
4: Set  $\text{dist}(s, s) = 0$ ,  $\sigma(s, s) = 1$ , and enqueue  $s$  into a queue  $Q$ ;
▼ Breadth-first traverse (BFT).
5: while  $Q$  is not empty do
6:   Dequeue  $v$  from  $Q$ , and then push  $v$  into  $S$ ;
7:   if Direction is 'Forward' then
8:     for  $w \in V_k$  satisfying that  $(v, w) \in E_k$  do
9:       Call Search_usingBFT( $s, v, w, Q, c, \sigma, \text{dist}, \text{adjacent}, \text{Direction}$ );
10:    end for
11:   end if
12:   if Direction is 'Backward' then
13:     for  $w \in V_k$  satisfying that  $(w, v) \in E_k$  do
14:       Call Search_usingBFT( $s, v, w, Q, c, \sigma, \text{dist}, \text{adjacent}, \text{Direction}$ );
15:    end for
16:   end if
17: end while

```

Algorithm 3 Search_usingBFT Function

Input: $s, v, w, Q, \sigma(*, *)$, $\text{dist}(*, *)$, $\text{adjacent}(*, *)$, Direction;

Output: Updated $Q, \sigma(*, *)$, $\text{dist}(*, *)$, $\text{adjacent}(*, *)$;

```

1: if  $\text{dist}(s, w) = \infty$  then
2:   ▼ First access.
3:   Insert  $w$  into  $Q$ ;
4: end if
5: if Direction is 'Forward' then
6:   if  $\text{dist}(s, w) > \text{dist}(s, v) + c(w)$  then
7:     ▼ SCP discovery.
8:     Update  $\text{dist}(s, w) = \text{dist}(s, v) + c(w)$ , set  $\sigma(s, w) = 0$  and  $\text{adjacent}(w) = \text{empty list}$ ;
9:   end if
10:  if  $\text{dist}(s, w) == \text{dist}(s, v) + c(w)$  then
11:    ▼ SCP counting.
12:    Compute  $\sigma(s, w) = \sigma(s, w) + \sigma(s, v)$ , and put  $v$  into  $\text{adjacent}(w)$ ;
13:  end if
14: end if
15: if Direction is 'Backward' then
16:   if  $\text{dist}(w, s) > \text{dist}(v, s) + c(v)$  then
17:     ▼ SCP discovery.
18:     Update  $\text{dist}(w, s) = \text{dist}(v, s) + c(v)$ , set  $\sigma(w, s) = 0$  and  $\text{adjacent}(w) = \text{empty list}$ ;
19:   end if
20:  if  $\text{dist}(w, s) == \text{dist}(v, s) + c(v)$  then
21:    ▼ SCP counting.
22:    Compute  $\sigma(w, s) = \sigma(w, s) + \sigma(v, s)$ , and put  $v$  into  $\text{adjacent}(w)$ ;
23:  end if
24: end if

```

update of BC_{SCP} on each vertex, which has the time complexity $O(|V|L)$. Consequently, the total time complexity is $O(\sum_{k=1}^L |V_k|(|E_k| + |V_k| \log(|V_k|)) + L^2 + |V|L)$, which is

Algorithm 4 Accumulate Function

Input: $s, t, w, \text{adjacent}(*), \sigma(*, *), d(*, *), d'(*, *), \eta(*), \theta(*), \text{Direction};$
Output: Updated $d(*, *), d'(*, *), \eta(*), \theta(*);$

▼ **Update $d(s; v)$ based on Equation (8).**

- 1: if Direction is 'Forward' then
- 2: for $v \in \text{adjacent}(w)$ do
- 3: Calculate $d(s; v) = d(s; v) + (\sigma(s, v)/\sigma(s, w)) \times (1 + d(s; w));$
- 4: end for
- 5: end if
- ▼ **Update $d'(t; v)$ based on Equation (9).**
- 6: if Direction is 'Backward' then
- 7: for $v \in \text{adjacent}(w)$ do
- 8: Calculate $d'(t; v) = d'(t; v) + (\sigma(v, t)/\sigma(w, t)) \times (1 + d'(t; w));$
- 9: end for
- 10: end if
- ▼ **Record $\eta(w)$, and $\theta(w)$.**
- 11: if $s == t_{in}^k$ then
- 12: Set $\eta(w) = d(s; w);$
- 13: end if
- 14: if $t == t_{out}^k$ then
- 15: Set $\theta(w) = d'(t; w);$
- 16: end if

significantly smaller than the time complexity $O(|V|(|E| + |V| \log(|V|)))$ of executing Dijkstra-algorithm [6] for BC_{SCP} on entire TDG.

Space complexity: The space complexity of Algorithm 1 is

$$O(|E| + |V|). \quad (11)$$

Specifically, (1) for G_k , sorting the neighbor table requires the space complexity $O(|E_k| + |V_k|)$. (2) Recording the SCP for each vertex needs the space complexity $O(|V_k|)$, which could be reused. (3) Storing $BC_{SCP}, \lambda, \alpha, \beta, \gamma$ for all vertices owns the space complexity $O(|V|)$. Consequently, the total space complexity is $O(|E| + |V|)$.

C Evaluation on Llama 3 Series

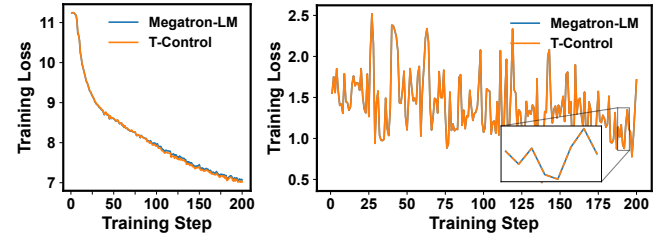
Since the baselines perform best on GPT 3 and Llama 2, we use these models for direct comparison in Section 6. Tables 7 and 8 show additional evaluations on the Llama 3 series, which represent the latest models. Notably, T-Control achieves the best performance compared with the state-of-the-art baselines.

Table 7. The performance (s/iteration) of different systems for the training of Llama 3-7B.

GPU Number	16	32
Megatron-LM (TP=1,PP=8,MBS=2)	12.91	6.89
Zero Bubble (TP=1,PP=8,MBS=2)	16.476	12.027
AdaPipe (TP=1,PP=4,MBS=2)	12.706	7.07
T-Control (TP=1,PP=4,MBS=2)	12.663	7.01

Table 8. The performance (s/iteration) of different systems for the training of Llama 3-70B.

GPU Number	80	160
Megatron-LM (TP=4,PP=20,MBS=1)	72.49	37.93
Zero Bubble (TP=4,PP=20,MBS=1)	OOM	OOM
AdaPipe (TP=4,PP=20,MBS=1)	failed	failed
T-Control (TP=4,PP=20,MBS=1)	50.93	29.82



(a) GPT 3-1.7B (64 GPUs).

(b) Llama 2-7B Lora (8 GPUs).

Figure 17. Training losses in different training processes.

D Robustness

To evaluate the robustness of T-Control, we compare its training loss with that of Megatron-LM. As shown in Figure 17, T-Control achieves almost identical training loss to both baselines. In the training of GPT 3-1.7B on 8 nodes with 64 GPUs, minor differences in the training curves arise due to non-deterministic accumulation orders in all-reduce communication. In contrast, for Llama 2-7B trained on a single node with 8 GPUs using data parallelism, the training curves of all methods nearly coincide.